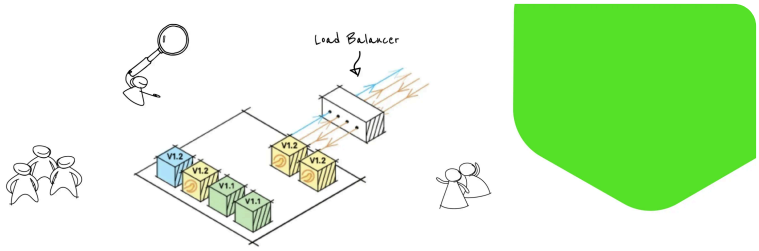
Unlocking DevEx in the Age of AI May 13 2026 | Virtual > Register Now

October 7, 2024

Intro To Deployment Strategies: Blue-Green, Canary, And More



Explore deployment strategies like blue-green and canary to enhance application reliability and reduce risks. Blue-green uses two environments to ensure smooth transitions, while canary gradually rolls out updates to minimize impact. Both strategies improve rollback capabilities and user experience, essential for modern DevOps practices.

Whether we mean to or not, software deployments look different across organizations, teams, and applications. This can make pushing the deployment button feel like playing a game of craps: you roll the dice and try to stay alive. Luckily, there are a few ways to limit the variance in success. This blog post will discuss the different strategies and practices that can help you succeed with your production deployments.

Deployment Strategies To Consider

Deployment strategies are practices used to change or upgrade a running instance of an application. The following sections will explain six deployment strategies. Let's start with discussing the basic deployment.

The Basic Deployment

In a basic deployment, all nodes within a target environment are updated at the same time with a new service or artifact version. Because of this, basic deployments are not outage-proof and they slow down rollback processes or strategies. Of all the deployment strategies shared, it is the riskiest.

Harness named a Leader in the 2025 Gartner® Magic Quadrant™ for DevOps Platforms

Read the report

Unlocking DevEx in the Age of AI May 13 2026 | Virtual > Register Now

Table of Contents

Deployment Strategies to Consider

The Basic Deployment

The Multi-Service Deployment

Rolling Deployment

Blue-Green Deployment

Canary Deployment

A/B Testing

Which Deployment Strategy Should I Use?

Eliminating After-Hours Deployments

Continuous Verification

Automate Your Deployments With Harness

Takeaways

Pros:

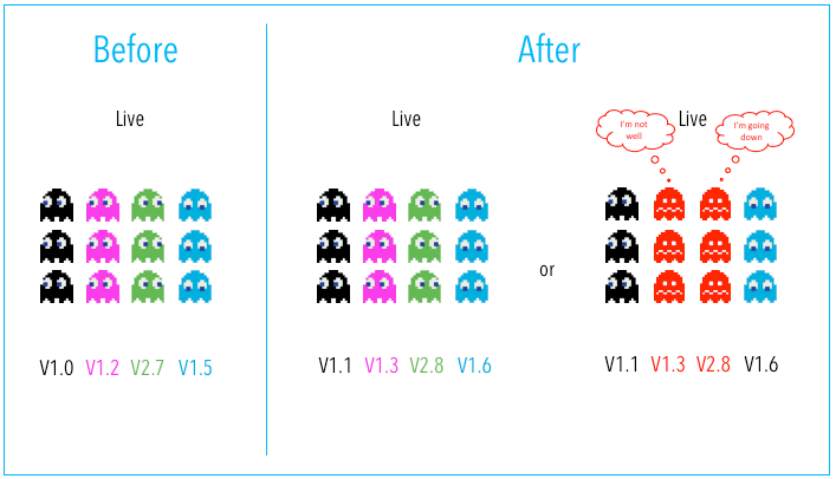
The benefits of this strategy are that it is simple, fast, and cheap. Use this strategy if 1) your application service is not business, mission, or revenue-critical, or 2) your deployment is to a lower environment, during off-hours, or with a service that is not in use.

Cons:

Of all the deployment strategies shared, it is the riskiest and does not fall into best practices. Basic deployments are not outage-proof and do not provide for easy rollbacks. When combined with manual processes, they are the greatest risk of a deployment disaster.

The Multi-Service Deployment

In a multi-service deployment, all nodes within a target environment are updated with multiple new services simultaneously. This strategy is used for application services that have service or version dependencies, or if you're deploying off-hours to resources that are not in use.



Pros:

Multi-service deployments are simple, fast, cheap, and not as risk-prone as a basic deployment.

Cons:

Multi-service deployments are slow to roll back and not outage-proof. Using this deployment strategy also leads to difficulty in managing, testing, and verifying all the service dependencies.

Rolling Deployment

A rolling deployment is a deployment strategy that updates running instances of an application with the new release. All nodes in a target environment are

Harness named a Leader in the 2025 Gartner® Magic Quadrant™ for DevOps Platforms

Read the report

SHARE -

Harness

Read more from this author

Next-Generation CI/CD For Dummies

Read the ebook

Book a Demo

Continuous Delivery & GitOps

Try it for Free

Continuous Delivery & GitOps

Unlocking DevEx in the Age of AI May 13 2026 | Virtual > Register Now

The benefits of a rolling deployment are that it is relatively simple to roll back, less risky than a basic deployment, and the implementation is simple.

Cons:

Since nodes are updated in batches, rolling deployments require services to support both new and old versions of an artifact. Verification of an application deployment at every incremental change also makes this deployment slow.

Blue-Green Deployment

Blue-green deployment is a deployment strategy that utilizes two identical environments, a “blue” (aka staging) and a “green” (aka production) environment with different versions of an application or service. Quality assurance and user acceptance testing are typically done within the blue environment that hosts new versions or changes. User traffic is shifted from the green environment to the blue environment once new changes have been testing and accepted within the blue environment. You can then switch to the new environment once the deployment is successful.

Pros:

One of the benefits of the blue-green deployment is that it is simple, fast, well-understood, and easy to implement. Rollback is also straightforward, because you can simply flip traffic back to the old environment in case of any issues. Blue-green deployments are therefore not as risky compared to other deployment strategies.

Cons:

Cost is a drawback to blue-green deployments. Replicating a production environment can be complex and expensive, especially when working with microservices. Quality assurance and user acceptance testing may not identify all of the anomalies or regressions either, and so shifting all user traffic at once can present risks. An outage or issue could also have a wide-scale business impact before a rollback is triggered, and depending on the implementation, in-flight user transactions may be lost when the shift in traffic is made.

Harness named a Leader in the 2025 Gartner® Magic Quadrant™ for DevOps Platforms

Read the report

Canary Deployment

A canary deployment is a deployment strategy that releases an application or service incrementally to a subset of users. All infrastructure in a target environment is updated in small phases (e.g: 2%, 25%, 75%, 100%). A canary release is the lowest risk-prone, compared to all other deployment strategies, because of this control.

Pros:

Unlocking DevEx in the Age of AI May 13 2026 | Virtual > Register Now

blue-green deployment because it does not require two production environments. And finally, it is fast and safe to trigger a rollback to a previous version of an application.

Cons:

Drawbacks to canary deployments involve testing in production and the implementations needed. Scripting a canary release can be complex: manual verification or testing can take time, and the required monitoring and instrumentation for testing in production may involve additional research.

A/B Testing

In A/B testing, different versions of the same service run simultaneously as "experiments" in the same environment for a period of time. Experiments are either controlled by feature flags toggling, A/B testing tools, or through distinct service deployments. It is the experiment owner's responsibility to define how user traffic is routed to each experiment and version of an application. Commonly, user traffic is routed based on specific rules or user demographics to perform measurements and comparisons between service versions. Target environments can then be updated with the optimal service version.

The biggest difference between A/B testing and other deployment strategies is that A/B testing is primarily focused on experimentation and exploration. While other deployment strategies deploy many versions of a service to an environment with the immediate goal of updating all nodes with a specific version, A/B testing is about testing multiple ideas vs. deploying one specific tested idea.

Pros:

A/B testing is a standard, easy, and cheap method for testing new features in production. And luckily, there are many tools that exist today to help enable A/B testing.

Cons:

The drawbacks to A/B testing involve the experimental nature of its use case. Experiments and tests can sometimes break the application, service, or user experience. Finally, scripting or automating AB tests can also be complex.

Which Deployment Strategy Should I Use?

Now that we know different deployment techniques, a commonly asked question may be, which deployment strategy should I use? The answer depends on the type of application you have and your target environment.

Based on conversations with Harness customers, most teams use blue-green or canary deployments for mission-critical web applications.

According to Thomas Wilson, SRE Manager at Burst SMS, "My CEO was next to me when an update brought our whole service down. He was shocked at how calm I was. Using Harness, we had everything back up in a couple of minutes. No harm done." Before Harness, outages could take two hours. Now, with blue/green deployments, that's down to under five minutes, a 96% improvement. You can read the whole case study.

Harness named a Leader in the 2025 Gartner® Magic Quadrant™ for DevOps Platforms

Read the report

Customers have minimal to little business impact when migrating from the blue-green deployment strategy to a canary deployment strategy. It's also common for teams to create their strategy based on combining the strategies we shared in this blog post. For example, some customers will do multi-service canary deployments.

Eliminating After-Hours Deployments

Unlocking DevEx in the Age of AI May 13 2026 | Virtual > Register Now

where it really matters is to leverage some deployment strategies and practices that can help with operationalizing our services.

- Some practices or standards to consider implementing include: service-specific deployment checklists
- Continuous Integration (CI) and Continuous Delivery (CD) tools
- Well-defined and understood environments
- Configuration management tools
- Communication channels like Slack
- An on-call or incident response playbook
- Automated rollbacks

A good portion of these practices can help with server or service downtime, software bugs, continuous feedback, and new application deployments. Aside from creating a foundation for better software delivery, there are also opportunities to leverage automation alongside our metrics and monitoring tools through the practice of Continuous Verification (CV).

Continuous Verification

Harness Continuous verification utilizes data and operationalizing tool stacks to take action based on the performance and quality of application deployment. With our customers who use Harness today, CV helps them with:

· Deployment Verification - inside the verify step of a deployment pipeline leading to auto-rollback and manual rollback as failure strategies.

· And 24x7 Service Guard - always-on, change impact analysis that measures overall service health and correlates it with deployments.

There can be many questions about operationalizing deployments across different tools, dependencies, and environments. Automating some of these challenges away is the next generation of scaling and simplifying software delivery.

Automate Your Deployments With Harness

If you are struggling to implement these advanced deployment and testing strategies with your current DevOps tools, click here to schedule a live demo with one of our experts and learn why hundreds of smart DevOps teams love Harness.

Takeaways

- Deployment strategies where both new and old versions of the application are running are safer.
- Automate the progressive rollout and rollback for your applications.
- Scripting advanced deployment strategies may be challenging. Modern continuous delivery tooling can help.

Harness named a Leader in the 2025 Gartner® Magic Quadrant™ for DevOps Platforms

Read the report

Unlocking DevEx in the Age of AI May 13 2026 | Virtual > Register Now

Harness

All this author's posts

Harness is a unified, end-to-end AI software delivery platform to manage the SDLC using purpose-built AI agents

Previous

The DevOps Tools Lifecycle Mesh for 2021

Next

Continuous Verification - Machine Learning to Safeguard Your Deployments

Similar Blogs

Canary Deployments And Operations In Kubernetes

In the Kubernetes ecosystem there are some lines in the sand that become blurry.

Continuous Delivery & GitOps

Advanced Deployment Strategies

This article talks about the different deployment strategies along with the...

Harness Platform

The Difference Between Rolling And Blue-Green Deployments

Understand the Rolling deployment and Blue-Green deployment methods to...

Feature Management & Experimentation

harness®

Harness named a Leader in the 2025 Gartner® Magic Quadrant™ for DevOps Platforms

Read the report

ess Updates

to receive the latest

ox

Submit

By submitting this form, you acknowledge and agree that Harness Inc will process your personal information in accordance with the Privacy Statement.

Unlocking DevEx in the Age of AI May 13 2026 | Virtual > Register Now

THE STATE OF DevOps Modernization 2026

Continuous Delivery & GitOps
Continuous Integration
Feature Management & Experimentation
Infrastructure as Code Management
Resilience Testing
AI SRE NEW
Database DevOps NEW
Artifact Registry
Internal Developer Portal
Software Engineering Insights
Application Security Testing
Web Application & API Protection
AI Security
Cloud Cost Management
AI Test Automation

Harness AI

Pricing
View plans

For Developers
Documentation
Developer Hub
Open Source
Commitment to Open Source
Community
API Reference Docs
Engineering Blog
Harness University
Training

Blog
Comparison Guide
Collateral
Customers
DevOps Academy
Ebooks
Engineering Maturity Assessment
On-demand Videos
Webinars
Support
Professional Services

About Us
Press & News
Partners
Careers
Contact Us
Trademark Policy
Security
Legal
Press Kit

© 2026 Harness Inc. Subscription Terms Website Terms of Use Privacy Statement Opt Out Cookie Settings

Harness named a Leader in the 2025 Gartner® Magic Quadrant™ for DevOps Platforms

Read the report